

Object-Oriented Programming with C++

Unit 3: Polymorphism — Detailed Practical Code Book

Programme: S.Y. B.Tech. Artificial Intelligence and Data Science

Semester: III

Course: Object-Oriented Programming with C++

Course Code: ADPC303

Unit: 3 — Polymorphism

Language Standard: C++17 or later

Code Book Purpose

This practical code book provides step-by-step, compilable C++ programs for Unit 3: Polymorphism.

The Unit 3 syllabus includes:

- Introduction to polymorphism.
- Types of polymorphism.
- Function overloading.
- Operator overloading.
- Unary and binary operator overloading.
- Run-time polymorphism.
- Base-class pointers.
- Virtual functions.
- Pure virtual functions.
- Virtual table concept.
- Virtual destructors.
- Abstract base classes.

How to Use This Code Book

For every Concept:

1. Read the aim and concept.
2. Study the syntax.
3. Type the program yourself.
4. Compile and run it.
5. Compare your result with the expected output.
6. Change one value and predict the result.
7. Complete the modification task.
8. Prepare the viva answers.

Compilation Commands

Linux/macOS

```
g++ -std=c++17 filename.cpp -o program  
./program
```

Windows with MinGW

```
g++ -std=c++17 filename.cpp -o program.exe  
program.exe
```

Index

Sr. No.	Program title	Main concept
1	Function overloading	Compile-time polymorphism
2	Area calculator	Function overloading with different parameters
3	Unary minus operator	Unary operator overloading
4	Prefix and postfix increment	Unary operator overloading
5	Complex number addition	Binary + operator overloading
6	Distance comparison	Relational operator overloading
7	Non-member/friend operator	Operator overloading using friend function
8	Base pointer without virtual function	Static binding demonstration
9	Base pointer with virtual function	Run-time polymorphism
10	Base reference with virtual function	Dynamic binding through references
11	Abstract class	Pure virtual function
12	Collection of shape pointers	Polymorphic processing
13	Virtual destructor	Safe deletion through base pointer
14	Object slicing	Why references/pointers are needed
15	Payment system	Abstract interface and real-world example
16	Payroll mini-project	Integrated polymorphism application

Concept 1: Function Overloading

Aim

To implement compile-time polymorphism using function overloading.

Concept

Function overloading means defining multiple functions with the same name but different parameter lists. The compiler selects the correct function based on the number, types, or order of arguments.

Syntax

```
returnType functionName(parameterList1);  
returnType functionName(parameterList2);
```

Program

```
#include <iostream>  
  
int add(int first, int second) {  
    return first + second;  
}  
  
double add(double first, double second) {  
    return first + second;  
}  
  
int add(int first, int second, int third) {  
    return first + second + third;  
}  
  
int main() {  
    std::cout << "Sum of two integers: " << add(10, 20) << '\n';  
    std::cout << "Sum of two doubles: " << add(2.5, 3.7) << '\n';  
    std::cout << "Sum of three integers: " << add(10, 20, 30) << '\n';  
}
```

```
    return 0;
}
```

Expected Output

Sum of two integers: 30
Sum of two doubles: 6.2
Sum of three integers: 60

Explanation

- `add(int, int)` is called for two integer values.
- `add(double, double)` is called for two decimal values.
- `add(int, int, int)` is called for three integer values.
- The return type alone cannot be used to overload a function.

Modification Task

Add an overloaded `add()` function that joins two `std::string` values.

Viva Questions

1. What is function overloading?
2. Which details can distinguish overloaded functions?
3. Can two functions differ only in return type?

Concept 2: Area Calculator Using Function Overloading

Aim

To calculate areas of different shapes using overloaded functions.

Program

```
#include <iostream>

int calculateArea(int side) {
    return side * side;
}

int calculateArea(int length, int width) {
    return length * width;
}

double calculateArea(double radius) {
    constexpr double PI = 3.141592653589793;
    return PI * radius * radius;
}

int main() {
    std::cout << "Square Area: " << calculateArea(5) << "\n";
    std::cout << "Rectangle Area: " << calculateArea(6, 4) << "\n";
    std::cout << "Circle Area: " << calculateArea(2.0) << "\n";

    return 0;
}
```

Expected Output

Square Area: 25
Rectangle Area: 24

Circle Area: 12.5664

Explanation

- One integer argument selects the square-area function.
- Two integer arguments select the rectangle-area function.
- One double argument selects the circle-area function.

Modification Task

Add an overloaded function for triangle area using base and height as double values.

Concept 3: Unary Minus Operator Overloading

Aim

To overload a unary minus operator for a user-defined class.

Concept

A unary operator works on one operand. In the expression `-number`, the operator function can define how negation should work for an object.

Program

```
#include <iostream>

class Number {
private:
    int value;

public:
    explicit Number(int givenValue) : value(givenValue) {}

    Number operator-() const {
        return Number(-value);
    }

    void display() const {
        std::cout << value << "\n";
    }
};

int main() {
    Number first(25);
    Number second = -first;

    std::cout << "Original value: ";
    first.display();

    std::cout << "Negated value: ";
    second.display();

    return 0;
}
```

Expected Output

Original value: 25
Negated value: -25

Key Point

The expression `-first` internally calls:
`first.operator-()`

Modification Task

Create a `Balance` class and overload unary minus to show the negative of the stored balance value.

Concept 4: Prefix and Postfix Increment Operator Overloading

Aim

To overload prefix and postfix increment operators.

Concept

- Prefix `++object`: increment first, then return updated object.
- Postfix `object++`: return old object, then increment.

A dummy `int` parameter distinguishes postfix overload from prefix overload.

Program

```
#include <iostream>

class Counter {
private:
    int value;

public:
    explicit Counter(int initialValue = 0) : value(initialValue) {}

    Counter& operator++() {
        ++value;
        return *this;
    }

    Counter operator++(int) {
        Counter old = *this;
        ++value;
        return old;
    }

    void display() const {
        std::cout << value << "\n";
    }
};

int main() {
    Counter counter(5);

    std::cout << "After prefix increment: ";
    ++counter;
    counter.display();

    std::cout << "Value returned by postfix increment: ";
    Counter oldValue = counter++;
```

```
oldValue.display();

std::cout << "Counter after postfix increment: ";
counter.display();

return 0;
}
```

Expected Output

After prefix increment: 6
Value returned by postfix increment: 6
Counter after postfix increment: 7

Syntax Reference

```
Counter& operator++();    // Prefix
Counter operator++(int);  // Postfix
```

Modification Task

Add prefix and postfix decrement operators to the same class.

Concept 5: Binary + Operator Overloading for Complex Numbers

Aim

To overload the binary + operator to add two complex numbers.

Program

```
#include <iostream>

class Complex {
private:
    int real;
    int imaginary;

public:
    Complex(int realPart = 0, int imaginaryPart = 0)
        : real(realPart), imaginary(imaginaryPart) {}

    Complex operator+(const Complex& other) const {
        return Complex(real + other.real, imaginary + other.imaginary);
    }

    void display() const {
        std::cout << real;
        if (imaginary >= 0) {
            std::cout << " + ";
        } else {
            std::cout << " - ";
        }
        std::cout << (imaginary >= 0 ? imaginary : -imaginary) << "i\n";
    }
};
```

```
int main() {  
    Complex first(2, 3);  
    Complex second(4, 5);  
    Complex sum = first + second;  
  
    std::cout << "First complex number: ";  
    first.display();  
  
    std::cout << "Second complex number: ";  
    second.display();  
  
    std::cout << "Sum: ";  
    sum.display();  
  
    return 0;  
}
```

Expected Output

First complex number: 2 + 3i
Second complex number: 4 + 5i
Sum: 6 + 8i

Explanation

first + second
↓
first.operator+(second)
↓
Returns a new Complex object

Modification Task

Overload the binary - operator to subtract two complex numbers.

Concept 6: Relational Operator Overloading

Aim

To overload the > operator for comparing two user-defined objects.

Program

```
#include <iostream>  
  
class Distance {  
private:  
    int meters;  
  
public:  
    explicit Distance(int value) : meters(value) {}  
  
    bool operator>(const Distance& other) const {  
        return meters > other.meters;  
    }  
  
    void display() const {  
        std::cout << meters << " meters\n";  
    }  
}
```

```
};

int main() {
    Distance first(120);
    Distance second(90);

    std::cout << "First distance: ";
    first.display();

    std::cout << "Second distance: ";
    second.display();

    if (first > second) {
        std::cout << "First distance is greater\n";
    } else {
        std::cout << "Second distance is greater or equal\n";
    }

    return 0;
}
```

Expected Output

First distance: 120 meters
Second distance: 90 meters
First distance is greater

Modification Task

Overload == to test whether two Distance objects store equal values.

Concept 7: Friend/Non-Member Operator Overloading

Aim

To overload an operator using a friend non-member function.

Concept

Sometimes the left operand is not an object of the current class. In `10 + complexNumber`, the left operand is an integer. A non-member friend function can implement this form.

Program

```
#include <iostream>

class Complex {
private:
    int real;
    int imaginary;

public:
    Complex(int realPart = 0, int imaginaryPart = 0)
        : real(realPart), imaginary(imaginaryPart) {}

    friend Complex operator+(int value, const Complex& number);

    void display() const {
        std::cout << real;
        if (imaginary >= 0) {
```

```
        std::cout << " + ";
    } else {
        std::cout << " - ";
    }
    std::cout << (imaginary >= 0 ? imaginary : -imaginary) << "i\n";
}
};
```

```
Complex operator+(int value, const Complex& number) {
    return Complex(value + number.real, number.imaginary);
}
```

```
int main() {
    Complex number(2, 3);
    Complex result = 10 + number;

    std::cout << "Result: ";
    result.display();

    return 0;
}
```

Expected Output

Result: 12 + 3i

Modification Task

Write a friend operator- that supports 10 - complexNumber.

Concept 8: Base Pointer Without a Virtual Function

Aim

To observe static binding when a base function is not virtual.

Program

```
#include <iostream>

class Base {
public:
    void display() const {
        std::cout << "Base display function\n";
    }
};

class Derived : public Base {
public:
    void display() const {
        std::cout << "Derived display function\n";
    }
};

int main() {
    Derived derivedObject;
    Base* basePointer = &derivedObject;

    basePointer->display();
}
```

```
    return 0;
}
```

Expected Output

Base display function

Explanation

basePointer has static type Base*. Since display() is not virtual, the compiler binds the call to Base::display().

Modification Task

Add a direct call derivedObject.display() and compare its output with basePointer->display().

Concept 9: Base Pointer With a Virtual Function

Aim

To implement run-time polymorphism using a virtual function and base pointer.

Program

```
#include <iostream>

class Animal {
public:
    virtual void sound() const {
        std::cout << "Animal makes a sound\n";
    }

    virtual ~Animal() = default;
};

class Dog : public Animal {
public:
    void sound() const override {
        std::cout << "Dog barks\n";
    }
};

class Cat : public Animal {
public:
    void sound() const override {
        std::cout << "Cat meows\n";
    }
};

int main() {
    Dog dog;
    Cat cat;

    Animal* animal = &dog;
    animal->sound();

    animal = &cat;
    animal->sound();

    return 0;
}
```

```
}
```

Expected Output

```
Dog barks  
Cat meows
```

Key Learning

A base pointer can point to different derived objects. A virtual function selects the correct derived implementation during execution.

Modification Task

Add a Cow class that displays Cow moos.

Concept 10: Base Reference With a Virtual Function

Aim

To use a base-class reference for run-time polymorphism.

Program

```
#include <iostream>

class Shape {
public:
    virtual double area() const {
        return 0.0;
    }

    virtual ~Shape() = default;
};

class Rectangle : public Shape {
private:
    double length;
    double width;

public:
    Rectangle(double givenLength, double givenWidth)
        : length(givenLength), width(givenWidth) {}

    double area() const override {
        return length * width;
    }
};

class Circle : public Shape {
private:
    double radius;

public:
    explicit Circle(double givenRadius) : radius(givenRadius) {}

    double area() const override {
        constexpr double PI = 3.141592653589793;
        return PI * radius * radius;
    }
}
```

```
};

void printArea(const Shape& shape) {
    std::cout << "Area: " << shape.area() << '\n';
}

int main() {
    Rectangle rectangle(5.0, 3.0);
    Circle circle(2.0);

    printArea(rectangle);
    printArea(circle);

    return 0;
}
```

Expected Output

```
Area: 15
Area: 12.5664
```

Key Learning

Passing objects by const Shape& avoids copying and preserves the real derived object for virtual dispatch.

Concept 11: Abstract Class and Pure Virtual Function

Aim

To create and use an abstract class.

Concept

A class with at least one pure virtual function is abstract. It cannot be instantiated directly.

Program

```
#include <iostream>

class Shape {
public:
    virtual double area() const = 0;
    virtual ~Shape() = default;
};

class Rectangle : public Shape {
private:
    double length;
    double width;

public:
    Rectangle(double givenLength, double givenWidth)
        : length(givenLength), width(givenWidth) {}

    double area() const override {
        return length * width;
    }
};

int main() {
```

```
Rectangle rectangle(8.0, 4.0);
std::cout << "Rectangle Area: " << rectangle.area() << '\n';
return 0;
}
```

Expected Output

Rectangle Area: 32

Key Learning

The declaration below makes Shape abstract:

```
virtual double area() const = 0;
```

Modification Task

Add a Triangle class and implement the formula $0.5 \times \text{base} \times \text{height}$.

Concept 12: Collection of Polymorphic Shape Pointers

Aim

To process different derived objects through a common abstract base interface.

Program

```
#include <iostream>
#include <memory>
#include <vector>

class Shape {
public:
    virtual double area() const = 0;
    virtual void displayName() const = 0;
    virtual ~Shape() = default;
};

class Rectangle : public Shape {
private:
    double length;
    double width;

public:
    Rectangle(double givenLength, double givenWidth)
        : length(givenLength), width(givenWidth) {}

    double area() const override {
        return length * width;
    }

    void displayName() const override {
        std::cout << "Rectangle";
    }
};

class Circle : public Shape {
private:
    double radius;
```

```
public:
    explicit Circle(double givenRadius) : radius(givenRadius) {}

    double area() const override {
        constexpr double PI = 3.141592653589793;
        return PI * radius * radius;
    }

    void displayName() const override {
        std::cout << "Circle";
    }
};

int main() {
    std::vector<std::unique_ptr<Shape>> shapes;
    shapes.push_back(std::make_unique<Rectangle>(5.0, 3.0));
    shapes.push_back(std::make_unique<Circle>(2.0));

    for (const auto& shape : shapes) {
        shape->displayName();
        std::cout << " Area: " << shape->area() << '\n';
    }

    return 0;
}
```

Expected Output

```
Rectangle Area: 15
Circle Area: 12.5664
```

Key Learning

- `std::unique_ptr<Shape>` safely owns derived objects through a base interface.
- A virtual destructor ensures correct cleanup.
- The loop calls the correct `area()` function for each actual object type.

Modification Task

Add a Triangle object to the vector.

Concept 13: Virtual Destructor

Aim

To demonstrate correct destruction of a derived object through a base pointer.

Program

```
#include <iostream>

class Base {
public:
    virtual ~Base() {
        std::cout << "Base destructor\n";
    }
};

class Derived : public Base {
public:
```

```
~Derived() override {
    std::cout << "Derived destructor\n";
}
};

int main() {
    Base* pointer = new Derived();
    delete pointer;
    return 0;
}
```

Expected Output

```
Derived destructor
Base destructor
```

Key Learning

When deleting through a base pointer, a virtual base destructor ensures that both derived and base destructors execute.

Better Modern Alternative

```
#include <memory>

std::unique_ptr<Base> pointer = std::make_unique<Derived>();
```

A virtual destructor is still required in a polymorphic base class.

Concept 14: Object Slicing Demonstration

Aim

To understand object slicing and avoid it using references.

Program

```
#include <iostream>

class Base {
public:
    virtual void display() const {
        std::cout << "Base object\n";
    }

    virtual ~Base() = default;
};

class Derived : public Base {
public:
    void display() const override {
        std::cout << "Derived object\n";
    }
};

void displayByValue(Base object) {
    object.display();
}

void displayByReference(const Base& object) {
```

```
    object.display();
}

int main() {
    Derived derived;

    std::cout << "Passing by value: ";
    displayByValue(derived);

    std::cout << "Passing by reference: ";
    displayByReference(derived);

    return 0;
}
```

Expected Output

Passing by value: Base object
Passing by reference: Derived object

Explanation

When derived is passed by value as a Base, only the base portion is copied. This is object slicing. Passing by reference preserves the complete derived object.

Modification Task

Write a third function using `const Base*` and call it with the address of the derived object.

Concept 15: Payment Processing System

Aim

To implement a real-world polymorphic payment system.

Program

```
#include <iostream>
#include <string>

class Payment {
public:
    virtual void pay(double amount) const = 0;
    virtual ~Payment() = default;
};

class CardPayment : public Payment {
public:
    void pay(double amount) const override {
        std::cout << "Paid Rs. " << amount << " using card\n";
    }
};

class UpiPayment : public Payment {
public:
    void pay(double amount) const override {
        std::cout << "Paid Rs. " << amount << " using UPI\n";
    }
};
```

```
class NetBankingPayment : public Payment {
public:
    void pay(double amount) const override {
        std::cout << "Paid Rs. " << amount << " using net banking\n";
    }
};

void processPayment(const Payment& payment, double amount) {
    payment.pay(amount);
}

int main() {
    CardPayment card;
    UpiPayment upi;
    NetBankingPayment netBanking;

    processPayment(card, 1250.0);
    processPayment(upi, 750.0);
    processPayment(netBanking, 500.0);

    return 0;
}
```

Expected Output

```
Paid Rs. 1250 using card
Paid Rs. 750 using UPI
Paid Rs. 500 using net banking
```

Concepts Used

- Abstract base class.
- Pure virtual function.
- Derived classes.
- Run-time polymorphism.
- Base reference.
- Virtual destructor.

Modification Task

Add a WalletPayment class and demonstrate it through processPayment().

Concept 16: Employee Payroll Mini-Project

Aim

To build a salary-calculation application using abstract classes and run-time polymorphism.

Program

```
#include <iostream>
#include <string>
#include <utility>

class Employee {
protected:
    int employeeId;
    std::string name;
```

```
public:
    Employee(int id, std::string employeeName)
        : employeeId(id), name(std::move(employeeName)) {}

    virtual double calculateSalary() const = 0;

    void displayBasicDetails() const {
        std::cout << "Employee ID: " << employeeId << "\n";
        std::cout << "Name: " << name << "\n";
    }

    virtual ~Employee() = default;
};

class PermanentEmployee : public Employee {
private:
    double basicSalary;
    double allowance;

public:
    PermanentEmployee(int id, std::string employeeName, double basic, double extra)
        : Employee(id, std::move(employeeName)), basicSalary(basic), allowance(extra) {}

    double calculateSalary() const override {
        return basicSalary + allowance;
    }
};

class ContractEmployee : public Employee {
private:
    double hourlyRate;
    int hoursWorked;

public:
    ContractEmployee(int id, std::string employeeName, double rate, int hours)
        : Employee(id, std::move(employeeName)), hourlyRate(rate), hoursWorked(hours) {}

    double calculateSalary() const override {
        return hourlyRate * hoursWorked;
    }
};

void printPaySlip(const Employee& employee) {
    employee.displayBasicDetails();
    std::cout << "Salary: Rs. " << employee.calculateSalary() << "\n\n";
}

int main() {
    PermanentEmployee permanentEmployee(101, "Asha", 40000.0, 8000.0);
    ContractEmployee contractEmployee(102, "Vikas", 500.0, 80);

    printPaySlip(permanentEmployee);
    printPaySlip(contractEmployee);

    return 0;
}
```

```
}
```

Expected Output

Employee ID: 101

Name: Asha

Salary: Rs. 48000

Employee ID: 102

Name: Vikas

Salary: Rs. 40000

Modification Task

1. Add a FreelanceEmployee class.
2. Add tax calculation for permanent employees.
3. Store employees using `std::vector<std::unique_ptr<Employee>>`.
4. Display the total payroll amount.

Quick Reference Sheet

Function overloading

```
int add(int a, int b);
```

```
double add(double a, double b);
```

Unary operator overloading

```
ClassName operator-() const;
```

```
ClassName& operator++();
```

```
ClassName operator++(int);
```

Binary operator overloading

```
ClassName operator+(const ClassName& other) const;
```

```
bool operator>(const ClassName& other) const;
```

Virtual function and override

```
class Base {
```

```
public:
```

```
    virtual void display() const;
```

```
    virtual ~Base() = default;
```

```
};
```

```
class Derived : public Base {
```

```
public:
```

```
    void display() const override;
```

```
};
```

Pure virtual function

```
virtual double area() const = 0;
```

Base pointer/reference

```
Base* pointer = &derivedObject;  
const Base& reference = derivedObject;
```

Safe polymorphic ownership

```
std::unique_ptr<Base> object = std::make_unique<Derived>();
```

Common Errors and Fixes

Error	Cause	Fix
Functions differ only by return type	Return type cannot distinguish overloads	Change the parameter list
override compilation error	Derived signature differs from base virtual function	Match name, parameters, const, and return type
Base version called through pointer	Base function is not virtual	Declare base function virtual
Derived cleanup missing	Base destructor is non-virtual	Use virtual ~Base() = default;
Derived behavior lost	Object passed by value as base type	Pass by reference or pointer
Operator meaning unclear	Overload is not intuitive	Use a named function instead
Cannot instantiate class	Class has pure virtual function	Instantiate a concrete derived class